

Tímový projekt
Virtual Data Analyst
Inteligentný systém pre analýzu dát pomocou
prirodzeného jazyka

Peter Muzslay, Janka Tarčáková, Jana Kampová, Jozef Guláš, Tamara Uhrinová

Akademický rok 2025/2026

Obsah

1	Úvod	3
1.1	Motivácia	3
1.2	Štruktúra projektu	3
2	Riešený problém	4
2.1	Definícia problému	4
2.2	Príklad použitia	4
3	Prístup a metodológia	5
3.1	Výber technológií	5
3.2	Architektúra GenAI Core	5
3.2.1	Query Generator	5
3.2.2	Response Summarizer	6
4	Architektúra systému	7
4.1	Vysokoúrovňová architektúra (High-Level)	7
4.1.1	Tok dát (Data Flow)	7
4.2	Technická architektúra (Low-Level)	8
4.2.1	Backend komponenty	8
4.2.2	API Endpoints	9
4.2.3	Bezpečnostné mechanizmy	9
4.2.4	Frontend komponenty	9
5	Výber a testovanie LLM modelov	10
5.1	Metodológia testovania	10
5.2	Porovnávané modely	10
5.3	Výsledky testovania – SQL generácia	10
5.4	Výsledky testovania – Sumarizácia	11
5.5	Záver výberu modelov	11
6	Použité technológie – detailný prehľad	12
6.1	Backend technológie	12
6.2	Frontend technológie	12
6.3	Infraštruktúra	12
6.4	AI/ML modely	12
7	Databázová vrstva	13
7.1	Počiatočný návrh databázovej vrstvy	13
7.2	Pripájanie na databázu	14
7.2.1	API endpointy pre správu pripojenia	14
8	Frontendová časť aplikácie	16
8.0.1	Štruktúra projektu	16
8.0.2	Správa stavu aplikácie	16
8.0.3	Vlastné React hooky	16
8.1	Komunikácia s backendom	17
8.1.1	Správa API volaní pomocou Axios	17
8.2	Používateľské rozhranie	18

8.2.1	Rozloženie aplikácie	18
8.2.2	Chatové rozhranie	19
8.2.3	Správa pripojenia k databáze	19
8.2.4	Modálne okno	20

1 Úvod

Projekt **Virtuálny dátový analytik** predstavuje moderný webový systém, ktorý umožňuje používateľom analyzovať dáta v databázach pomocou prirodzeného jazyka. Využitím pokročilých techník spracovania prirodzeného jazyka (NLP) a veľkých jazykových modelov (LLM) dokáže systém transformovať otázky v bežnom jazyku na SQL dotazy, vykonať ich nad databázou a následne vrátiť zrozumiteľné odpovede.

Hlavným cieľom projektu je demokratizácia prístupu k dátovej analytike – umožniť aj netechnickým používateľom efektívne získavať informácie z databáz bez potreby znalosti SQL alebo iných programovacích jazykov.

1.1 Motivácia

V súčasnej dobe je dátová analytika kľúčovým faktorom pri rozhodovaní v mnohých oblastiach podnikania. Avšak tradičné nástroje pre prácu s databázami vyžadujú:

- Znalosť SQL jazyka
- Pochopenie štruktúry databázy
- Technické zručnosti pre interpretáciu výsledkov

Virtuálny dátový analytik tieto bariéry odstraňuje a sprístupňuje dátovú analýzu širšiemu okruhu používateľov.

1.2 Štruktúra projektu

Projekt je rozdelený do nasledujúcich hlavných komponentov:

1. **Frontend** – Používateľské rozhranie (React + Vite)
2. **Backend** – Serverová aplikácia (FastAPI, Python)
3. **GenAI Core** – Jadro pre spracovanie prirodzeného jazyka (LLM)
4. **Databázová vrstva** – Pripojenie a správa databáz

2 Riešený problém

2.1 Definícia problému

Hlavný problém, ktorý projekt rieši, je **preklad prirodzeného jazyka na štruktúrované databázové dotazy** a následná **interpretácia výsledkov** pre koncového používateľa.

Konkrétne výzvy zahŕňajú:

- **Sémantické porozumenie** – Pochopenie zámeru používateľa z neformálnej otázky
- **Mapovanie na schému** – Správne priradenie entít v otázke k tabuľkám a stĺpcom databázy
- **Generovanie validného SQL** – Syntakticky a sémanticky správne dotazy
- **Bezpečnosť** – Zabezpečenie len čítacích operácií (READ-ONLY)
- **Zrozumiteľná odpoveď** – Sumarizácia výsledkov v prirodzenom jazyku

2.2 Príklad použitia

Vstup (používateľ)	Výstup (systém)
„Koľko objednávok bolo vytvorených v novembri 2019?“	Vygenerovaný SQL dotaz, počet záznamov a textová sumarizácia výsledkov
„Aký je priemerný počet položiek na objednávku?“	Analytický výsledok s vysvetlením
„Ukáž mi top 10 produktov podľa predaja“	Zoradený zoznam s popisom

Tabuľka 1: Príklady interakcie s virtuálnym analytikom

3 Prístup a metodológia

3.1 Výber technológií

Pri návrhu systému sme zvolili nasledujúce technológie:

Komponent	Technológia	Zdôvodnenie
Frontend	React + Vite	Moderný, reaktívny framework s rýchlym vývojovým prostredím
Backend	FastAPI (Python)	Asynchrónne API s automatickou dokumentáciou
Databáza	PostgreSQL	Robustná open-source relačná databáza
LLM	GPT-4.1 / Gemini 2.5	Najmodernejšie jazykové modely pre NLP úlohy
Cache	Redis	In-memory databáza pre rýchle ukladanie do vyrovnávacej pamäte
Kontajnerizácia	Docker	Konzistentné prostredie pre vývoj a nasadenie

Tabuľka 2: Použité technológie a ich zdôvodnenie

3.2 Architektúra GenAI Core

Jadro systému pre spracovanie prirodzeného jazyka pozostáva z troch hlavných komponentov:

1. **Router (LLM)** – Rozhoduje o spôsobe spracovania dotazu
2. **Query Generator** – Generuje SQL dotazy z prirodzeného jazyka
3. **Response Summarizer** – Sumarizuje výsledky pre používateľa

3.2.1 Query Generator

Komponent využíva model GPT-4.1 pre generovanie SQL dotazov. Systémový prompt definuje pravidlá:

- Generovať len **SELECT** dotazy (read-only prístup)
- Nikdy nepoužívať INSERT, UPDATE, DELETE, DROP, CREATE a iné modifikačné príkazy
- Vrátiť len čistý SQL dotaz bez vysvetlení
- Používať správnu PostgreSQL syntax

3.2.2 Response Summarizer

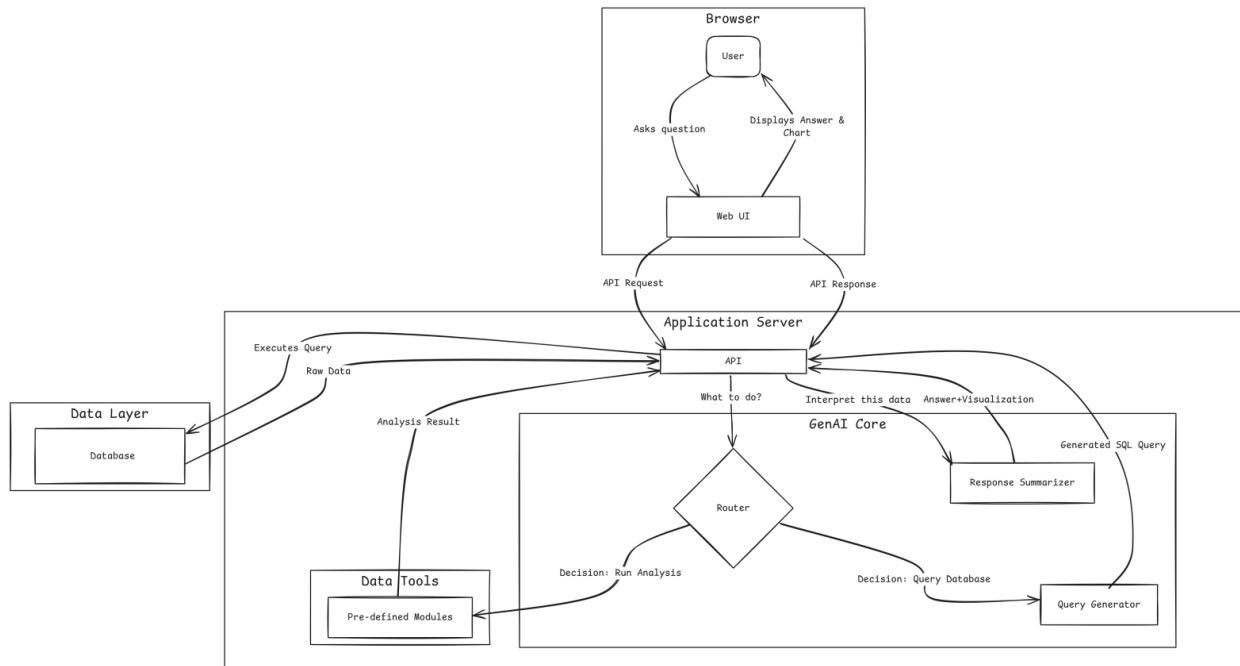
Komponent využíva model Gemini 2.5 Flash pre sumarizáciu výsledkov. Poskytuje:

- Popis, čo dáta zobrazujú
- Zvýraznenie kľúčových poznatkov a vzorcov
- Zmienku o významných hodnotách alebo trendoch
- Zrozumiteľný text pre netechnických používateľov

4 Architektúra systému

4.1 Vysokoúrovňová architektúra (High-Level)

Systém je navrhnutý ako **trojvrstvová architektúra** s jasne oddelenými zodpovednosťami:

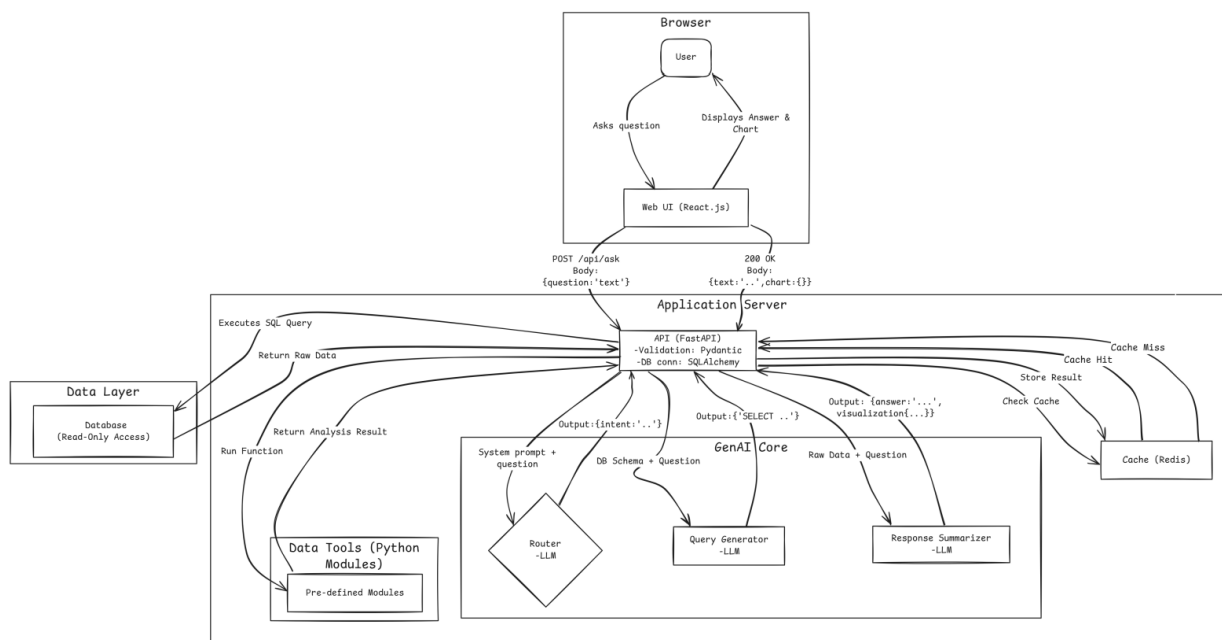


Obr. 1: Vysokoúrovňová architektúra systému

4.1.1 Tok dát (Data Flow)

1. Používateľ zadá otázku v prirodzenom jazyku cez webové rozhranie
2. Frontend odošle požiadavku na Backend API (/api/ask)
3. Backend získa schému databázy
4. Query Generator (GPT-4.1) vygeneruje SQL dotaz
5. Dotaz sa vykoná nad databázou (READ-ONLY)
6. Response Summarizer (Gemini 2.5) vytvorí textovú sumarizáciu
7. Odpoveď sa vráti používateľovi

4.2 Technická architektúra (Low-Level)



Obr. 2: Detailná architektúra systému s komponentmi

4.2.1 Backend komponenty

Komponent	Popis
DatabaseManager	Správa pripojenia k PostgreSQL, vynucovanie READ-ONLY módu, získavanie schémy
QueryGenerator	LLM-based generovanie SQL dotazov pomocou GPT-4.1
ResponseSummarizer	LLM-based sumarizácia výsledkov pomocou Gemini 2.5 Flash
Router	(TODO) Rozhodovanie medzi Data Tools a Database queries
RedisClient	(TODO) Caching pre optimalizáciu opakovaných dotazov

Tabuľka 3: Backend komponenty a ich zodpovednosti

4.2.2 API Endpoints

Endpoint	Metóda	Popis
/api/ask	POST	Hlavný endpoint – otázka → SQL → sumarizácia
/api/generate-sql	POST	Generovanie SQL bez vykonania
/api/query-and-summarize	POST	Vykonanie SQL a sumarizácia
/api/connect-database	POST	Pripojenie k databáze
/api/disconnect-database	POST	Odpojenie databázy
/api/schema	GET	Získanie schémy databázy

Tabuľka 4: REST API endpoints

4.2.3 Bezpečnostné mechanizmy

Systém implementuje viacúrovňové zabezpečenie pre READ-ONLY prístup:

1. **LLM Prompt** – Systémový prompt zakazuje generovanie modifikačných príkazov
2. **Regex validácia** – Kontrola nebezpečných kľúčových slov (INSERT, UPDATE, DELETE, DROP...)
3. **SQL Session** – Nastavenie SET TRANSACTION READ ONLY na úrovni databázy

4.2.4 Frontend komponenty

Komponent	Popis
HomePage	Hlavná stránka aplikácie, správa stavu pripojenia
Chat	Chat rozhranie pre komunikáciu s analytikom
SideBar	Bočný panel s ovládacími prvkami
DatabaseModal	Modal pre zadanie pripojovacích údajov k databáze
InputQuestion	Vstupné pole pre zadávanie otázok
UserMessage	Zobrazenie správ v chate

Tabuľka 5: React komponenty frontendu

5 Výber a testovanie LLM modelov

Pri návrhu systému bolo kľúčové rozhodnúť, ktoré veľké jazykové modely (LLM) použiť pre jednotlivé úlohy. Keďže systém vyžaduje dva odlišné typy spracovania – generovanie SQL dotazov a sumarizáciu výsledkov – rozhodli sme sa pre špecializovaný prístup s využitím rôznych modelov.

5.1 Metodológia testovania

Pre objektívne porovnanie výkonnosti LLM modelov sme vytvorili testovací skript, ktorý automatizovane vyhodnocoval schopnosti modelov riešiť SQL problémy. Ako zdroj testovacích úloh sme použili problémy z platformy **HackerRank**, ktorá poskytuje široké spektrum SQL výziev rôznej obtiažnosti.

Testovací proces zahŕňal:

1. **Kolekcia problémov** – Výber 50+ SQL problémov z HackerRank (easy, medium, hard)
2. **Automatizované testovanie** – Skript generoval dotazy pomocou každého modelu
3. **Validácia výsledkov** – Porovnanie vygenerovaných dotazov s očakávanými výstupmi
4. **Meranie metrík** – Presnosť, čas odozvy, konzistencia výstupov

5.2 Porovnávané modely

Model	Poskytovateľ
GPT-4.1	OpenAI/Azure
GPT-4o	OpenAI/Azure
GPT-3.5-turbo	OpenAI/Azure
Gemini 2.5 Flash	Google
Gemini 2.5 Pro	Google
Claude 4 Sonnet	Anthropic

Tabuľka 6: Porovnávané LLM modely a ich testované úlohy

5.3 Výsledky testovania – SQL generácia

Pre generovanie SQL dotazov sme testovali modely na nasledujúcich kritériách:

- **Syntaktická správnosť** – Percentuálna úspešnosť generovania validného SQL
- **Sémantická presnosť** – Či dotaz vracia očakávané výsledky
- **Dodržiavanie pravidiel** – Generovanie len READ-ONLY dotazov
- **Konzistencia** – Stabilita výstupov pri opakovaných dotazoch

GPT-4.1 dosiahol dobré výsledky v syntaktickej aj sémantickej presnosti, pričom dosiahol aj dodržiavanie READ-ONLY pravidiel. Napriek vyššej latencii a mierne vyššej cene sme ho zvolili pre kritickú úlohu generovania SQL.

5.4 Výsledky testovania – Sumarizácia

Pre sumarizáciu výsledkov sme hodnotili:

- **Zrozumiteľnosť** – Kvalita textu pre netechnických používateľov
- **Faktická správnosť** – Presnosť interpretácie dát
- **Rýchlosť** – Čas potrebný na generovanie odpovede
- **Cena** – Náklady na API volania

Gemini 2.5 Flash ponúkol optimálny pomer kvality, rýchlosti a ceny. Pri minimálnom rozdiely v kvalite oproti drahším modelom poskytuje výrazne nižšiu latenciu, náklady a veľké kontextové okno.

5.5 Záver výberu modelov

Na základe testovania sme sa rozhodli pre kombináciu:

- **GPT-4.1 (Azure)** pre generovanie SQL: dobrá presnosť a spoľahlivosť pri dodržiavaní bezpečnostných pravidiel
- **Gemini 2.5 Flash (Vertex AI)** pre sumarizáciu: optimálny pomer veľkosť/cena-/rýchlosť

6 Použité technológie – detailný prehľad

6.1 Backend technológie

- **Python 3.12+** – Hlavný programovací jazyk
- **FastAPI** – Moderný, asynchrónny webový framework
- **SQLAlchemy** – ORM a databázové pripojenie
- **OpenAI SDK** – Klient pre LLM API (GPT-4.1, Gemini)
- **Pydantic** – Validácia dát a schémy
- **uv** – Moderný Python package manager

6.2 Frontend technológie

- **React** – JavaScript knižnica pre UI
- **Vite** – Rýchly build nástroj
- **Tailwind CSS** – Utility-first CSS framework
- **Axios** – HTTP klient pre API volania

6.3 Infraštruktúra

- **Docker** – Kontajnerizácia aplikácií
- **Docker Compose** – Orchestrácia multi-kontajnerových aplikácií
- **Redis** – In-memory cache

6.4 AI/ML modely

- **GPT-4.1 (Azure)** – Generovanie SQL dotazov
- **Gemini 2.5 Flash (Vertex AI)** – Sumarizácia výsledkov

7 Databázová vrstva

Databázová vrstva predstavuje jednu zo základných súčastí navrhovanej aplikácie, keďže zabezpečuje ukladanie, spracovanie a poskytovanie dát pre ostatné komponenty systému.

7.1 Počiatočný návrh databázovej vrstvy

Databázovú časť sme v úvodnej fáze vývoja navrhli tak, aby poskytovala dostatočný základ pre testovanie aplikácie a prácu s dátami. Použili sme kontajnerové riešenie, vďaka ktorému bolo možné databázu rýchlo spustiť a jednoducho integrovať do vývojového prostredia. Kontajner umožnil opakovateľné nasadenie rovnakého databázového prostredia, čo výrazne uľahčilo overovanie správnej funkcionality prepojenia s backendom.

Ako prvú databázu sme zvolili PostgreSQL, ktorá je stabilná a široko používaná relačná databáza. Štruktúra databázy vychádzala z verejne dostupného datasetu z platformy **Kaggle**, ktorý obsahuje údaje o správaní zákazníkov v e-commerce prostredí.

Pred samotným použitím sme dataset upravili tak, aby bolo možné dáta efektívne využívať počas vývoja a testovania aplikácie. Konkrétne sme vykonali nasledujúce úpravy:

- odstránili sme neúplné záznamy, ktoré by mohli negatívne ovplyvniť testovanie,
- zjednotili sme dátové typy pre zjednodušenie importu a spracovania dát,
- pripravili sme dataset na automatický import do databázy.

V inicializačnom SQL skripte sme vytvorili tabuľku **events**, ktorá obsahuje kľúčové stĺpce:

- **event_id** – jedinečný identifikátor udalosti,
- **event_time** – čas vzniku udalosti,
- **event_type** – typ udalosti,
- **product_id**, **category_id**,
- **category_code**, **brand**,
- **price**, **user_id**,
- **user_session**.

Súčasťou skriptu bolo aj vytvorenie indexov pre rýchle vyhľadávanie podľa času udalosti, typu udalosti, produktu a používateľa. Tento prístup umožňuje efektívnu prácu s databázou aj pri väčších objemoch dát. Dáta sa automaticky načítavali zo súboru CSV pri štarte databázového kontajnera, takže boli hneď pripravené na testovanie. Takto pripravená databáza umožnila overiť správnu komunikáciu backendu s databázou, testovať základné operácie s dátami a kontrolovať korektnosť ich spracovania. Napriek tomu, že nešlo o finálnu databázovú vrstvu, predstavovala vhodný základ pre ďalší vývoj aplikácie.

V tejto fáze bola databáza súčasťou rovnakého projektu **Docker Compose** ako backend a frontend, čo umožňovalo spúšťanie všetkých služieb naraz a výrazne zjednodušovalo integráciu a úvodné testovanie aplikácie.

S postupujúcim vývojom aplikácie sme sa rozhodli databázu oddeliť do samostatného súboru **Docker Compose**. Toto riešenie umožňuje databázu spúšťať a spravovať nezávisle od backendu a frontendu, zjednodušuje jej samostatné testovanie a poskytuje väčšiu kontrolu nad konfiguráciou databázového prostredia, napríklad pri zmene verzie databázy. Oddelenie databázy zároveň vytvára priestor pre ďalšie úpravy a plánované rozšírenia aplikácie o podporu rôznych typov databáz.

7.2 Pripájanie na databázu

Po vytvorení a inicializácii databázového prostredia bolo ďalším krokom návrh a implementácia aplikačnej vrstvy, ktorá umožňuje backendu dynamicky sa pripájať k databáze.

Databázový konektor bol navrhnutý ako samostatná logická súčasť backendu, ktorá sprostredkúva komunikáciu medzi aplikáciou a databázou PostgreSQL. Cieľom bolo umožniť flexibilné pripojenie k rôznym databázovým inštanciam bez nutnosti meniť kód aplikácie. Hlavné vlastnosti konektora:

- bol implementovaný ako modul vo FastAPI,
- využíva štandardné PostgreSQL knižnice na nadväzovanie spojenia,
- podporuje dynamické vytváranie pripojenia na základe vstupov používateľa,
- uchováva informáciu o aktívnom pripojení počas behu aplikácie.

7.2.1 API endpointy pre správu pripojenia

Na ovládanie databázového spojenia sme vytvorili dva samostatné API endpointy:

1. Endpoint na pripojenie k databáze

- umožňuje používateľovi nadviazať spojenie s databázou,
- prijíma prihlasovacie údaje v tele požiadavky,
- po úspešnom pripojení vracia potvrdenie o nadviazaní spojenia.

2. Endpoint na odpojenie od databázy

- slúži na bezpečné ukončenie aktívneho pripojenia,
- používateľ ho aktivuje jednoduchým kliknutím (alebo volaním endpointu),
- zabezpečuje uvoľnenie zdrojov na strane backendu.

Pri vytváraní spojenia používateľ zadáva nasledujúce parametre:

- Host – adresa databázového servera,
- User – meno databázového používateľa,
- Password – prístupové heslo k databáze,
- Database name – meno cieľovej databázy,
- Port – port, na ktorom databáza počúva.

Tieto údaje sú zasielané backendu, ktorý ich následne spracuje. Na základe zadaných údajov aplikácia automaticky vytvorí tzv. connection string. Tento reťazec sa následne použije na inicializáciu spojenia s databázou prostredníctvom databázovej knižnice.

Podporované typy pripojenia

Aplikácia umožňuje pripojenie na dva typy databázových prostredí:

- Lokálnu databázu
 - bežiacu priamo na počítači vývojára alebo serveri,
 - vhodnú najmä na vývoj a testovanie.
- Databázu spustenú v Docker kontajneri
 - bežiacu na pozadí ako samostatná služba,
 - prístupnú cez názov kontajnera alebo localhost,
 - využívanú najmä pri integrácii s ostatnými službami v Docker Compose.

Odpojenie databázy

Proces odpojenia je riešený jednoducho:

- používateľ aktivuje endpoint na odpojenie,
- backend uzavrie aktívne spojenie,
- systém uvoľní pridelené zdroje,
- aplikácia prejde do stavu „nepripojená“.
- Tento mechanizmus zabraňuje zbytočnému držaniu otvorených spojení.

Po úspešnom pripojení k databáze aplikácia umožňuje:

- načítať zoznam dostupných tabuliek,
- zobrazíť stĺpce vybranej tabuľky,
- vypísať názvy stĺpcov spolu s ich dátovými typmi.

8 Frontendová časť aplikácie

8.0.1 Štruktúra projektu

Frontendová časť aplikácie je implementovaná pomocou knižnice React v kombinácii s nástrojom Tailwind. Projekt je rozdelený do modulov, aby bol prehľadný a komponenty sa dali ľahko znovu použiť. Komponenty sú navrhnuté tak, aby boli čo najviac nezávislé a komunikovali medzi sebou prostredníctvom vlastností (props).

Adresárová štruktúra projektu je nasledovná:

- **components/** - Komponenty, ktoré tvoria používateľské rozhranie, napríklad chat, bočný panel, modálne okná a vstupné polia.
- **hooks/** - Vlastné React hooky, ktoré riešia logiku aplikácie, napríklad pripojenie k databáze.
- **services/** - Funkcie na komunikáciu s backendom.
- **pages/** - Komponenty pre hlavné stránky aplikácie.
- **assets/** - Obrázky, ikony a iné statické súbory.
- **api/** - Nastavenie HTTP klienta pre volania na backend.

8.0.2 Správa stavu aplikácie

Správa stavu aplikácie je realizovaná pomocou vstavaných React hookov `useState`, `useEffect` a `useRef`.

Stavy sú spravované lokálne v jednotlivých komponentoch, napríklad:

- otvorenie a zatvorenie modálneho okna pre databázu,
- stav načítavania počas pripájania databázy,
- vstupné hodnoty formulárových polí,
- priebeh komunikácie v chatovom rozhraní.

8.0.3 Vlastné React hooky

Vo fronte je použitý vlastný React hook `useDatabase`, ktorý sa stará o pripojenie a odpojenie databázy. Komponenty ho môžu použiť bez toho, aby sa museli starať o detaily HTTP komunikácie.

Hook `useDatabase`:

- pripája a odpojuje databázu cez backend,
- sleduje, či sa práve načítava (`loading`),
- uchováva a zobrazuje chyby (`error`),

8.1 Komunikácia s backendom

Frontendová aplikácia komunikuje s backendom prostredníctvom HTTP požiadaviek. Táto komunikácia zabezpečuje pripojenie a odpojenie databázy, ako aj odosielanie otázok používateľa a prijímanie odpovedí od systému. Frontend odosiela požiadavky na backend vo formáte JSON a backend vracia odpovede v rovnakom formáte.

8.1.1 Správa API volaní pomocou Axios

Pre komunikáciu s backendom je použitá knižnica **Axios**. V projekte je vytvorený jeden centrálny Axios klient, ktorý obsahuje základnú konfiguráciu, ako je adresa backendového servera a hlavičky požiadaviek.

Listing 1: Základná konfigurácia Axios klienta

```
1 import axios from "axios";
2
3 const api = axios.create({
4   baseURL: import.meta.env.VITE_API_BASE_URL || "http://localhost
5     :8000",
6   headers: {
7     "Content-Type": "application/json",
8   },
9 });
10 export default api;
```

Na základe tohto klienta sú vytvorené servisné funkcie, ktoré reprezentujú jednotlivé API volania.

Listing 2: Servisné funkcie pre komunikáciu s backendom

```
1 import api from "./api";
2
3 export const connectDatabase = async (data) => {
4   const response = await api.post("/api/connect-database", data);
5   return response.data;
6 };
7
8 export const disconnectDatabase = async () => {
9   const response = await api.post("/api/disconnect-database");
10  return response.data;
11 };
12
13 export const askQuestion = async (question) => {
14   const response = await api.post("/api/ask", {
15     question,
16   });
17   return response.data;
18 };
```

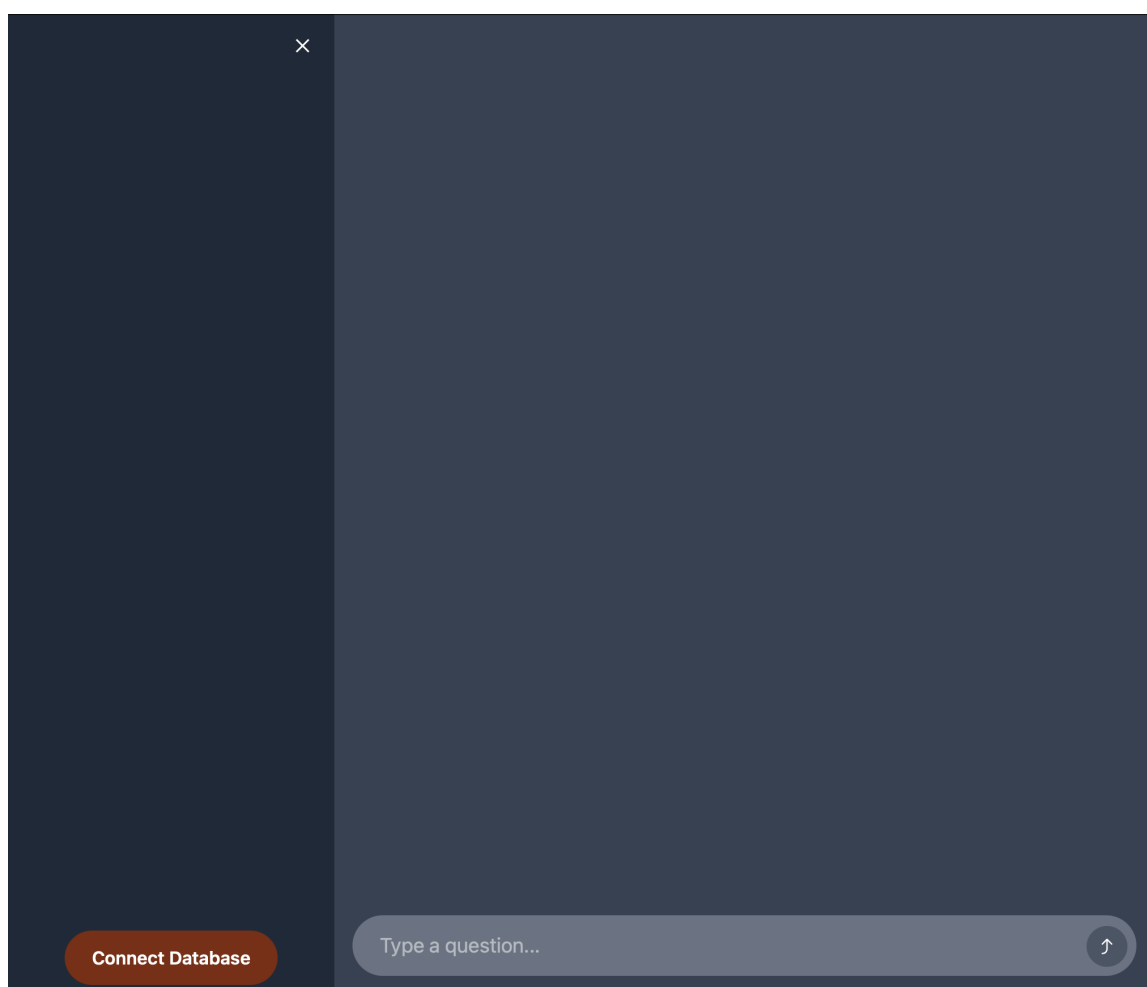
8.2 Používateľské rozhranie

Frontendová aplikácia má jednoduché a prehľadné rozhranie, ktoré umožňuje používateľovi komunikovať so systémom a spravovať pripojenie k databáze.

8.2.1 Rozloženie aplikácie

Hlavná stránka je rozdelená na dve časti:

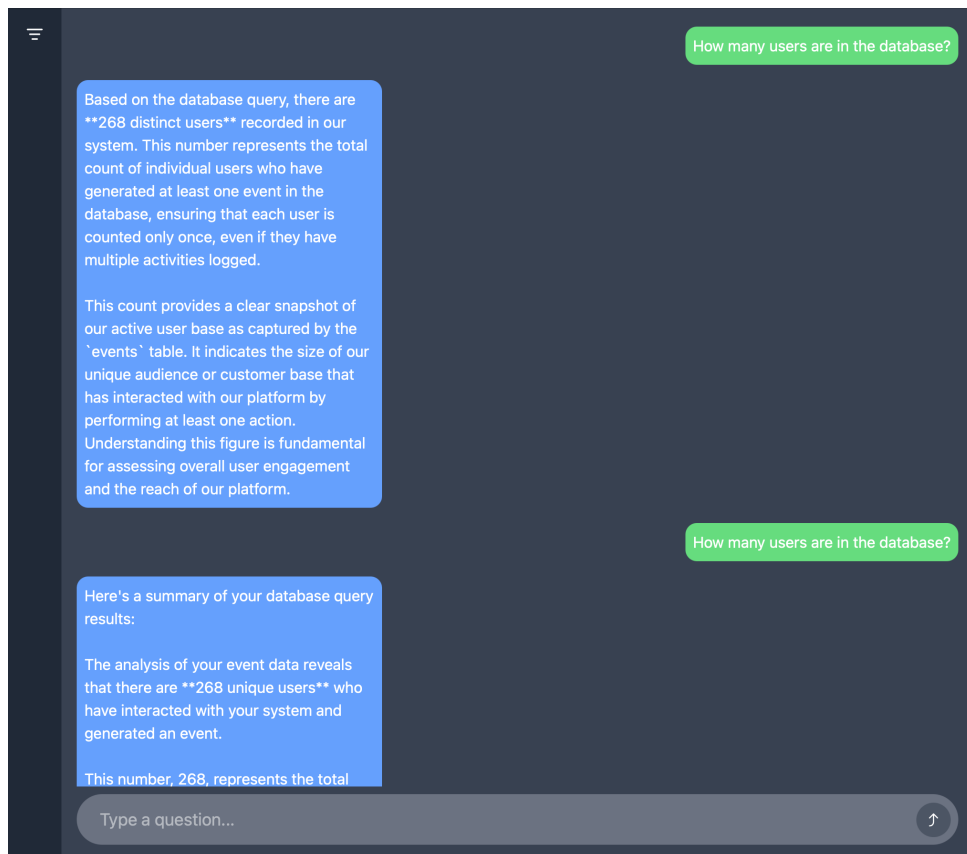
- **Bočný panel (Sidebar)** - obsahuje tlačidlá na otvorenie modálu pre pripojenie k databáze a ďalšie ovládacie prvky.
- **Hlavný obsah (Chat)** - zobrazujú sa správy medzi používateľom a systémom.



Obr. 3: Ukážka rozloženia frontendovej aplikácie

8.2.2 Chatové rozhranie

Chat pozostáva z oblasti správ a vstupného poľa na zadávanie otázok. Používateľ môže zadávať otázky, ktoré sú posielané systému, a výsledky sa zobrazujú v chronologickom poradí.

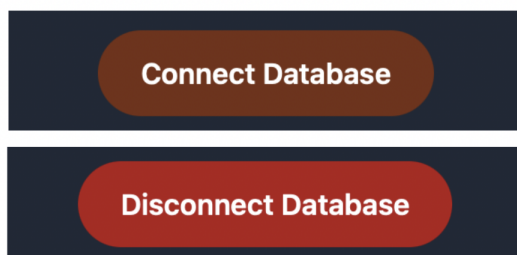


Obr. 4: Ukážka chatového rozhrania

8.2.3 Správa pripojenia k databáze

Používateľ môže databázu pripojiť alebo odpojiť pomocou tlačidla v bočnom paneli. Stav pripojenia sa vizuálne zobrazuje priamo na tlačidle:

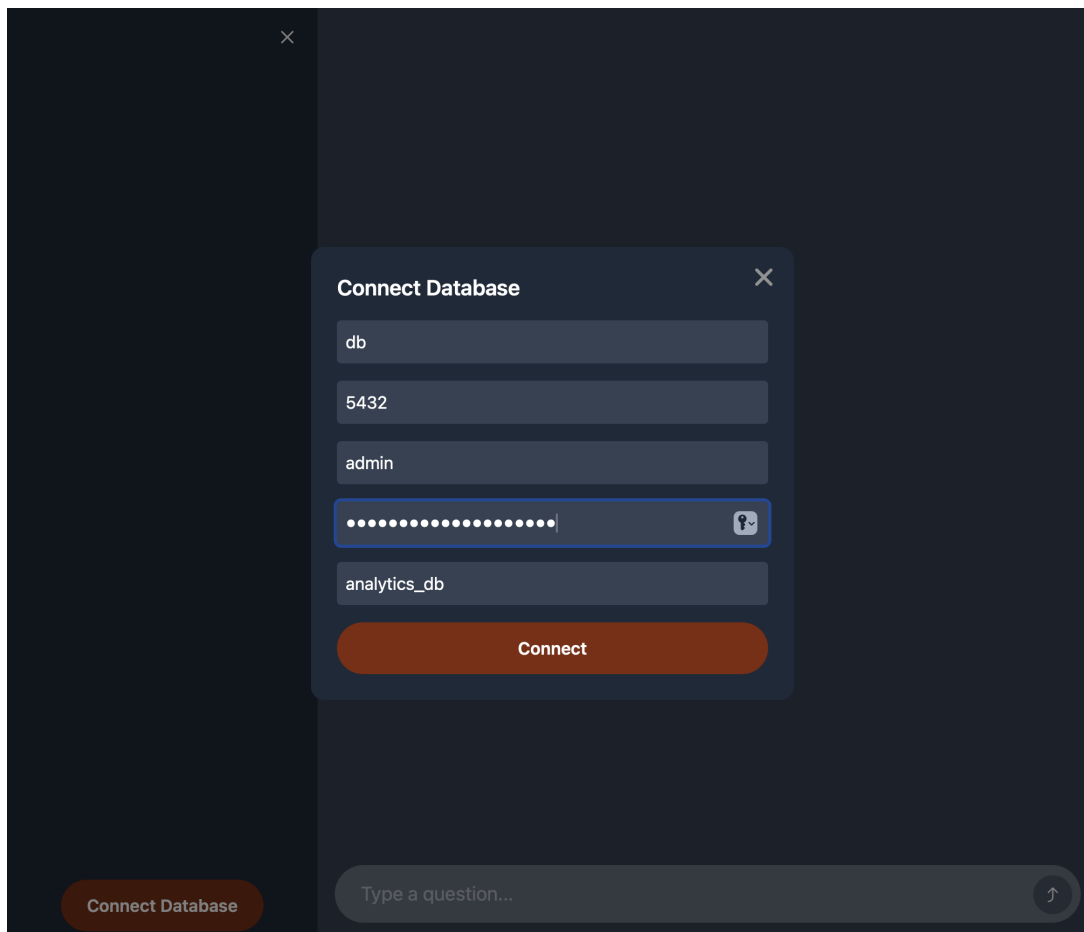
- **Connect Database** - databáza nie je pripojená,
- **Disconnect Database** - databáza je pripojená.



Obr. 5: Stav pripojenia k databáze zobrazený na tlačidle

8.2.4 Modálne okno

Modálne okno slúži na zadanie údajov potrebných na pripojenie k databáze, ako sú hostiteľ, port, používateľské meno, heslo a názov databázy. Počas pripájania sa používateľovi zobrazuje stav **Connecting...**, aby vedel, že sa proces práve vykonáva. V prípade chyby sa zobrazí chybová hláška, ktorá informuje o neúspešnom pripojení.



Obr. 6: Ukážka modálneho okna pre pripojenie databázy

Použitie nástrojov umelej inteligencie

Pri vypracovaní projektu sme využili nasledujúce AI nástroje ako podporné prostriedky:

- **Claude Opus 4.5 (Anthropic, 2025):** Využitý pri návrhu architektúry systému, code review a optimalizácii kódu.
- **Claude Sonnet 4.5 (Anthropic, 2025):** Využitý pri návrhu architektúry systému, code review a optimalizácii kódu. Pomáhal s formuláciou systémových promptov pre LLM komponenty.
- **GPT-4.1 (OpenAI/Azure, 2025):** Integrovaný priamo v systéme ako Query Generator pre transformáciu prirodzeného jazyka na SQL dotazy.
- **Gemini 2.5 Flash (Google, 2025):** Integrovaný v systéme ako Response Summarizer pre generovanie zrozumiteľných textových sumarizácií výsledkov dotazov.

Zdroje

1. FastAPI Documentation. (2025). *FastAPI - Modern, Fast Web Framework*.
<https://fastapi.tiangolo.com/>
2. React Documentation. (2025). *React - A JavaScript Library for Building User Interfaces*.
<https://react.dev/>
3. OpenAI. (2025). *GPT-4 API Documentation*.
<https://platform.openai.com/docs/>
4. Google Cloud. (2025). *Vertex AI - Gemini API*.
<https://cloud.google.com/vertex-ai/docs/generative-ai/>
5. PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*.
<https://www.postgresql.org/docs/>
6. SQLAlchemy Documentation. (2025). *SQLAlchemy - The Database Toolkit for Python*.
<https://docs.sqlalchemy.org/>
7. Docker Documentation. (2025). *Docker Docs*.
<https://docs.docker.com/>
8. Redis Documentation. (2025). *Redis Documentation*.
<https://redis.io/documentation>
9. Medium (2023). *Building Fast and Robust APIs with FastAPI and SQL Databases*.
<https://medium.com/towards-data-engineering/fastapi-with-sql-1c7852ccbf21>
10. Budibase. (2023). *How to Integrate Multiple Databases*.
<https://budibase.com/blog/data/how-to-integrate-multiple-databases/>
11. Vite Documentation. (2025). *Vite - Next Generation Frontend Tooling*.
<https://vitejs.dev/>
12. Tailwind CSS Documentation. (2025). *Tailwind CSS - A Utility-First CSS Framework*.
<https://tailwindcss.com/docs>
13. Axios Documentation. (2025). *Axios - Promise Based HTTP Client*.
<https://axios-http.com/docs/intro>
14. React Hooks Documentation. (2025). *React Hooks - State and Lifecycle in Functional Components*.
<https://react.dev/reference/react/hooks>
15. Brown, T., et al. (2020). *Language Models are Few-Shot Learners*.
<https://arxiv.org/abs/2005.14165>
16. Rajkumar, N., et al. (2022). *Evaluating the Text-to-SQL Capabilities of Large Language Models*.
<https://arxiv.org/abs/2204.00498>

17. Pourreza, M., & Rafiei, D. (2024). *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL*.
<https://arxiv.org/abs/2304.11015>